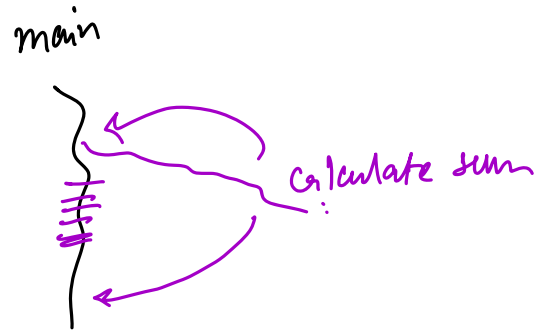


Agenda

- ① Callables / Futures.
- ② Multithreaded Merge Sort
- ③ Introduction to synchronisation problem
- ④ Solutions for synchronisation

→ Limitation of Runnable?

```
public interface Runnable {  
    void run();  
}
```



```
public interface Callable<T> {  
    T call() throws Exception;  
}
```

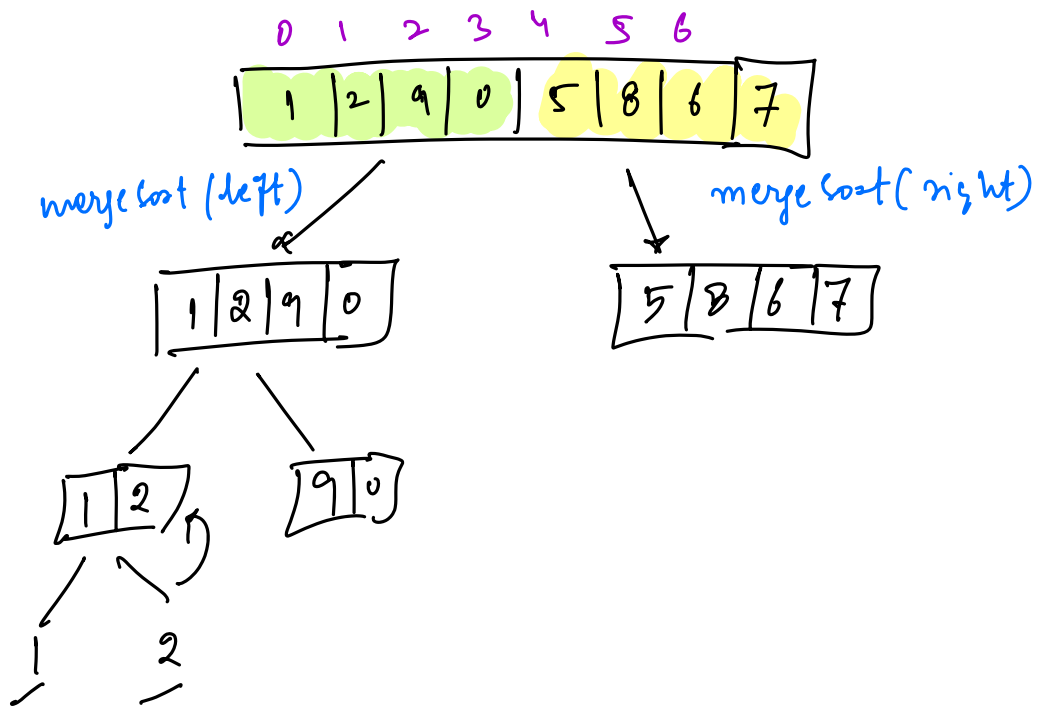
- ① return type T — the call() method returns some value.
- ② It can throw exceptions.

<T> in Callable <T> → generic type
↓
concrete impl. will define this.

Callable <Integer>

Callable <String>

→ Merge Sort (Divide and Conquer)



$$X = 0$$

→ Adder thread: ~~+1~~, ~~+2~~, ~~+3~~, ... ~~+100~~

→ Subtractor thread: ~~-1~~, ~~-2~~, ~~-3~~, ... ~~-100~~

$$X = 0$$

Adder

count.val += i

3 steps

- ① Read count.val
- ② Updating temp = count.val + i
- ③ Write count.val = temp.

Subtractor

count.val -= i

+1, -1

val = ~~0~~ 1

Adder (value += 1)

- ① Read val → 0
- local copy → 0

preempt
via context switching

preempt

- ② increment := 0 + 1 = 1
- ③ write to memory →
value in memory = 1

Subtractor (value -= 1)

- ① Read val → 0
local copy → 0
- ② decrement := 0 - 1 = -1
- ③ write := write -1 to memory
= ~~0~~

expected → +1 - 1 = 0
Actual → +1

} Subtraction is lost

Booking Show

Person A Thread $\rightarrow$ AS available $\rightarrow$ Read Yes $\checkmark$

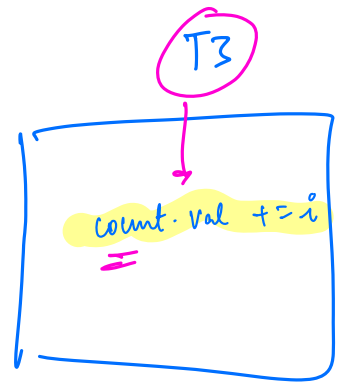
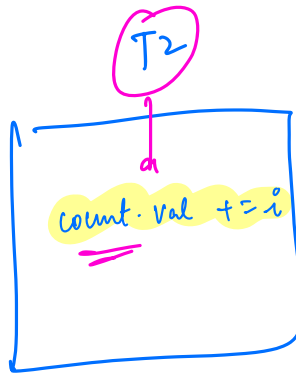
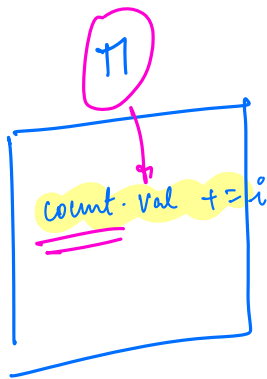
Person B . . .

Result $\rightarrow$ Two people book same seat $\rightarrow$ X

$\rightarrow$ Critical Section.

$\left. \begin{array}{l} \text{count.val} += i \\ \text{count.val} -= i \end{array} \right\}$

$\rightarrow$ Race Condition



$\rightarrow$ Pre-emptiveness (Preempt)

→ Mutex → exclusion.
↓
mutual

T1
Adder

lock().

value += i

 → read, write
unlock()

T2

Subtractor

try to enter
lock() → required.
waiting

time ↓

lock() → acquire

value -= i

 →
unlock();